National University of Singapore
School of Computing
CS3244: Machine Learning
Solution to Tutorial 8

Decision Trees and Ensemble Methods

Readings: *CIML, Chapter 1 (Decision Trees) & Chapter 13 (Ensemble Methods).*

1. **Decision Trees.**

   The loans department of DBN (Development Bank of NUS) has the following past loan processing records each containing an applicant's income, credit history, debt, and the final approval decision.

Income	Credit History	Debt	Decision
0 – 5K	Bad	Low	Reject
0 – 5K	Good	Low	Approve
0 – 5K	Unknown	High	Reject
0 – 5K	Unknown	Low	Approve
0 – 5K	Unknown	Low	Approve
0 – 5K	Unknown	Low	Reject
5 – 10K	Bad	High	Reject
5 – 10K	Good	High	Approve
5 – 10K	Unknown	High	Approve
5 – 10K	Unknown	Low	Approve
Over 10K	Bad	Low	Reject
Over 10K	Good	Low	Approve

   (a) **Random Forest.** Construct a random forest of size 3, where each of the three decision trees is fully grown from two of the three attributes, again based on the same set of examples: {Income, Credit History}, {Credit History, Debt} and {Debt, Income}.

   Based on the earlier answers for the decision tree, we know that Credit History will be the root node as it has a better information gain. This leads to pure "Bad" and "Good" leaves, leading to only using the secondary attribute for the remaining six cases of "Unknown".

   The {Income, Credit History} tree is identical to the one for the full decision tree, except that the bottom Debt node is left off and replaced by the mode of the examples that fulfill the tests (Credit History == "Unknown" ∩ Income == "0 – 5K"). This is a tie, so the parent node's test of just Credit History == "Unknown" is used to obtain "Approve".

   For the {Credit History, Debt} tree, the Credit History == "Unknown" takes a Debt test, resulting "Approve" for "Low" values (by the mode of examples), and also "Approve"

for "High" values (by the mode of the parent, since the mode of the examples is tied).

For the {Debt, Income} tree, we can recall that the information gain of the previous decision tree showed that Income was better than Debt. All three subcases of Income are not pure and each needs a test for Debt. In the "0 – 5K" Income, Debt == "Low" yields "Approve" by the mode of the examples, and yields "Reject" for "High" Debt cases. In the "5 – 10K" Income, Debt == "High" yields "Approve" by the mode of the examples, and yields "Approve" by purity when Debt is "Low". In the "Over 10K" Income, there are no examples of "High", so it defaults to "Approve". Similarly, since there is a tied outcome for Debt == "Low", we default to the mode of all examples, hence "Approve".

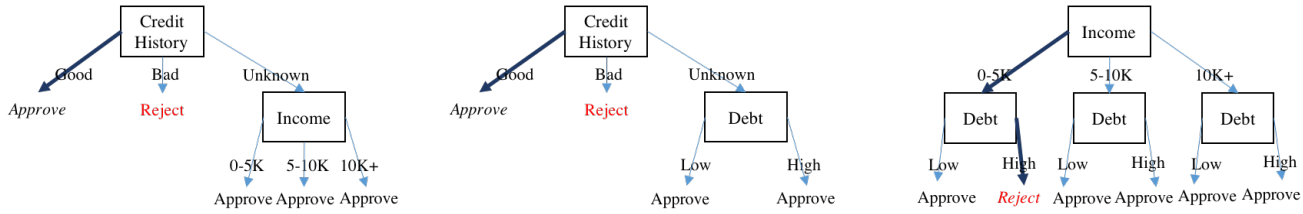


Figure 1: Random Forest of size 3 in Part (c). The dark edges and italicized labels correspond to the decision for each tree for the individual described in Part (d).

2. **Analyzing Adaboost**

Algorithm 1 AdaBoost for Binary Classification

1: Initialize the data points weights to $\theta^{(1)} = [\frac{1}{m}, \frac{1}{m}, ..., \frac{1}{m}]$
2: **for** $t = 1, 2, ..., T$ **do**
3: Fit a classifier $g_t(x)$ to the training data with weights $\theta^{(t)}$
4: Compute the $\theta^{(t)}$- weighted 0/1 error ϵ_t by:

$$\epsilon_t = \frac{\sum_{i=1}^{m} \theta_i^{(t)} I(y_i \neq g_t(x_i))}{\sum_{i=1}^{m} \theta_i^{(t)}}$$

where $I(x)$ is a function that output 1 if x is true, 0 otherwise.
5: Compute $\alpha_t = \frac{1}{2} \ln(\frac{1-\epsilon_t}{\epsilon_t})$
6: Set $\theta_i^{(t+1)} \leftarrow \frac{\theta_i^{(t)} exp[-\alpha_t y_i g_t(x_i)]}{Z_t}$, $i = 1, 2, ..., m$ where Z_t is the normalizer in t-th iteration.
7: **return** $G(x) = sign[\sum_{t=1}^{T} \alpha_t g_t()x]$

From the above algorithm, we have:

$$\theta_i^{(t+1)} = \frac{\theta_i^{(t)} \exp\left[-\alpha_t y^{(i)} h_t\left(x^{(i)}\right)\right]}{Z_t}$$

Therefore, we have:

$$\begin{aligned}\theta_i^{(T+1)} &= \theta_i^{(1)} \frac{\exp\left[-\alpha_1 y^{(i)} h_1\left(x^{(i)}\right)\right]}{Z_1} \times \cdots \times \frac{\exp\left[-\alpha_T y^{(i)} h_T\left(x^{(i)}\right)\right]}{Z_T} \\ &= \frac{1}{N} \frac{\exp\left[-y^{(i)} \sum_t \alpha_t h_t\left(x^{(i)}\right)\right]}{\prod_t Z_t} \quad \left(\text{Since } \theta_i^1 = \frac{1}{N}\right)\end{aligned}$$

From this we have:

$$\begin{aligned}
\text{Training Error}(H) &= \frac{1}{N}\sum_i I\left(y^{(i)} \neq H\left(x^{(i)}\right)\right) \text{ (Where } I \text{ is the Indicator Function.)} \\
&= \frac{1}{N}\sum_i I\left(y^{(i)} G\left(x^{(i)}\right) \leq 0\right) \\
&\leq \frac{1}{N}\sum_i \exp\left(-y^{(i)} G\left(x^{(i)}\right)\right) \\
&\text{Since } e^{-z} \geq 1 \text{ If } z \leq 0 \\
&= \sum_i \theta_i^{(T+1)} \times \prod_t Z_t \\
&= \prod_t Z_t
\end{aligned}$$

Note that The max value of $I(z) = 1$. Since $z \leq 0$, and we replace the indicator function with the exponential, the equality holds.

Note that $\sum_i \theta_i^{(T+1)} = 1$ since θ_T is a distribution

So we have defined an **upper bound** for the training error. However, it it not clear yet whether this upper bound decreases over multiple iterations and converges to a limit.

Actually we can further derive the upper bound of the training error that depends on ϵ_t(the miss-classification rate of the tth classifier; ranging from 0 to 1).

Below, we given you three options. **Which is the best upper bound for the training error?** (one of them is not a correct upper bound, one of them is a looser upper bound, and the final one is the best upper bound)

A. $\exp\left(-2\sum_{t=1}^{T}\gamma_t^1\right)$

B. $\exp\left(-2\sum_{t=1}^{T}\gamma_t^2\right)$

C. $\exp\left(-2\sum_{t=1}^{T}\gamma_t^3\right)$

HINT: Calculate Z_t, where $Z_t = \sum_i \theta_i^{(t)} \exp\left[-\alpha_t y^{(i)} h_t\left(x^{(i)}\right)\right]$

Also, you can use $(1 - \frac{1}{x})^x \leq \frac{1}{e}$

Let's see how can we bound Z_t. If we find tighter upper bounds for Z_t, then we can find tighter bounds for the training error. Also we

$$
\begin{aligned}
Z_t &= \sum_{i}^{m} \theta_i^{(t)} \exp\left[-\alpha_t y^{(i)} h_t\left(x^{(i)}\right)\right] \\
&= \sum_{i:y_i=h(x_i)} \theta_i^{(t)} e^{-\alpha_t} + \sum_{i:y_i\neq h(x_i)} \theta_i^{(}t) e^{\alpha_t} \\
&= (1-\epsilon_t)e^{-\alpha_t} + \epsilon_t.e^{\alpha_t} \\
(1-\epsilon_t).\frac{1}{\sqrt{\frac{1-\epsilon_t}{\epsilon_t}}} + \epsilon_t.\sqrt{\frac{1-\epsilon_t}{\epsilon_t}} &\left(\textit{Use } \alpha_t = \frac{1}{2}.ln(\frac{1-\epsilon_t}{\epsilon_t})\right) \\
&= 2\sqrt{\epsilon_t(1-\epsilon_t)} \\
&= 2\sqrt{(\frac{1}{2}-\gamma_t)(\frac{1}{2}+\gamma_t)} \\
&= \sqrt{1-4\gamma_t^2} \\
&= \left(1-\frac{2\gamma_t^2}{\frac{1}{2}}\right)^{\frac{1}{2}} \\
&\leq e^{-2\gamma_t^2} \ \textit{Since } (1-\frac{1}{x})^x \leq e^{-1}
\end{aligned}
$$

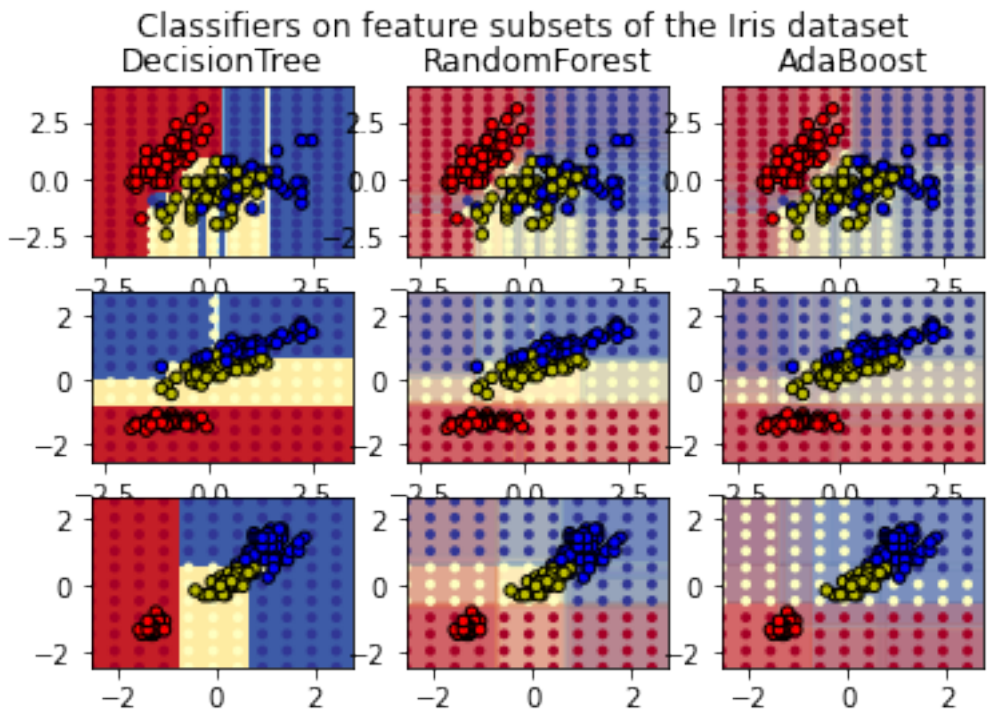
Now we have a bound on every Z_t, we can have a tighter bound on the training error in total

$$
\begin{aligned}
\textit{Training error} &\leq \prod_{t=1}^{T} Z_t \\
&\leq \prod_{t=1}^{T} e^{-2\gamma_t^2} \\
&\leq exp(-2\sum_{t=1}^{T}\gamma_t^2)
\end{aligned}
$$

3. **Visualizing Decision Tree Boundaries**

   Let's compare the decision boundaries of Decision Trees, Random Forests and the AdaBoost classifiers with Decision Stumps to classify on the Iris dataset.

   For the sake of visualization, we will choose just two features from the set of 5 features of Iris dataset. Each experiment will make use of exactly two features. We will experiment with these 3 pairs of features: 0 and 1, 1 and 2, 0 and 2.



(a) From the visualization above, is Random Forest doing better than Decision Tree? Will the result still be the same, or improve if we have less estimators (say 10) for Random Forest?

   i. From the diagram, RF and DT both have same performance.

   ii. If the number of estimators increase both RF and DT perform better and if the number of estimators decrease then RF and DT perform worse.

(b) The vizualization used a seed of 13. What if the seed was changed to 119? Will the performance of three classifiers, decision tree, random forest and adaboost will still be same? If not, why?

   i. DT and Adaboost will remain the same but RF may change. This is because DT and Adaboost use all the training data. However, RF might do bootstrpping and does not use all the data.

4. **Uniform Blending.** One of the simplest ensemble methods is Uniform Blending. Given a set of hypothesis: $h_1, h_2, h_3, ..., h_T$, Uniform blending makes predictions simply by mixing the predictions given by $h_1, h_2, h_3, ..., h_T$ uniformly. Concretely, for binary classification, uniform blending predicts by:

$$H(x) = \text{sign}(\sum_{t=1}^{T} h_t(x)), \tag{1}$$

and for regression, uniform blending predicts by:

$$H(x) = \frac{1}{T}\sum_{t=1}^{T} h_t(x) \tag{2}$$

Taking regression as an example, show that the performance (measured by out-of-sample error) of uniform blending is no worse than the average performance over $h_1, h_2, h_3, ..., h_T$ i.e.

$$\frac{1}{T}\sum_{t=1}^{T} J_{\text{test}}(h_t(x)) \geq J_{\text{test}}(H(x)) \tag{3}$$

Assume we evaluate the testing error by mean square error.

Hint: Start by calculating the average error over $h_1, h_2, h_3, ..., h_T$ for one fixed data point x.

Proving equation 3 can give us an intuition on why ensembling can help to reduce the out-of-sample error.

Let's first see the case for only one fixed data point x. Suppose the ground truth function is $f(x)$. The average error over $h_1, h_2, h_3, ..., h_T$ for point x is

$$\begin{aligned}
\frac{1}{T}\sum_{t=1}^{T}[h_t(x) - f(x)]^2 &= \frac{1}{T}\sum_{t=1}^{T}\left[h_t(x)^2 - 2h_t(x)f(x) + f(x)^2\right] && (4)\\
&= \frac{1}{T}\sum_{t=1}^{T} h_t(x)^2 - \frac{2f(x)}{T}\sum_{t=1}^{T} h_t(x) + \frac{1}{T}\sum_{t=1}^{T} f(x)^2 && (5)\\
&= \frac{1}{T}\sum_{t=1}^{T} h_t(x)^2 - 2f(x)H(x) + \frac{1}{T}\sum_{t=1}^{T} f(x)^2 \qquad H(x) = \tfrac{1}{T}\textstyle\sum_{t=1}^{T} h_t(x) && (6)\\
&= \frac{1}{T}\sum_{t=1}^{T} h_t(x)^2 - 2f(x)H(x) + f(x)^2 \qquad f(x) \text{ does not depend on } t && (7)\\
&= \frac{1}{T}\sum_{t=1}^{T} h_t(x)^2 - H(x)^2 + H(x)^2 - 2f(x)H(x) + f(x)^2 && (8)\\
&= \frac{1}{T}\sum_{t=1}^{T} h_t(x)^2 - H(x)^2 + [H(x) - f(x)]^2 && (9)\\
&= \frac{1}{T}\sum_{t=1}^{T} h_t(x)^2 - 2H(x)^2 + H(x)^2 + [H(x) - f(x)]^2 && (10)\\
&= \frac{1}{T}\sum_{t=1}^{T} h_t(x)^2 - \frac{1}{T}\sum_{t=1}^{T} 2h_t(x)H(x) + \frac{1}{T}\sum_{t=1}^{T} H(x)^2 + [H(x) - f(x)]^2 && (11)\\
&= \frac{1}{T}\sum_{t=1}^{T}\left[h_t(x)^2 - 2h_t(x)H(x) + H(x)^2\right] + [H(x) - f(x)]^2 && (12)\\
&= \frac{1}{T}\sum_{t=1}^{T}[h_t(x) - H(x)]^2 + [H(x) - f(x)]^2 && (13)
\end{aligned}$$

To compute the out-of-sample error, we only need to compute the expected error over all x,

hence, the average out-of-sample error over $g_1, g_2, g_3, ..., g_T$ *is*

$$\frac{1}{T}\sum_{t=1}^{T} J_{test}(h_t(x)) = \frac{1}{T}\sum_{t=1}^{T} \mathbb{E}_x\left\{[h_t(x) - f(x)]^2\right\} \tag{14}$$

$$= \mathbb{E}_x\left\{\frac{1}{T}\sum_{t=1}^{T} [h_t(x) - f(x)]^2\right\} \tag{15}$$

$$= \mathbb{E}_x\left\{\frac{1}{T}\sum_{t=1}^{T} [h_t(x) - H(x)]^2 + [H(x) - f(x)]^2\right\} \tag{16}$$

$$= \mathbb{E}_x\left\{\frac{1}{T}\sum_{t=1}^{T} [h_t(x) - H(x)]^2\right\} + \mathbb{E}_x\left\{[H(x) - f(x)]^2\right\} \tag{17}$$

$$= \mathbb{E}_x\left\{\frac{1}{T}\sum_{t=1}^{T} [h_t(x) - H(x)]^2\right\} + J_{test}(H(x)) \tag{18}$$

$$\geq J_{test}(H(x)) \tag{19}$$

From the above deduction we can see, the average out-of-sample error over $h_1, h_2, h_3, ..., h_T$ *is greater than or equal to the out-of-sample error of* $H(x)$*. Hence, the performance (measured by out-of-sample error) of uniform blending is no worse than the average performance over* $h_1, h_2, h_3, ..., h_T$*.*